

Core model: A Project owns durable keyed Artifacts and provenance. Keys, not row order, align related Artifacts. `project.translate(...)` always returns an output-label->Artifact mapping. Structural `project.*` methods create durable lineage-aware Artifacts; `artifact.analysis.*` and `artifact.visualize.*` do not extend the graph.

START / OPEN / IMPORT

```
import text_analysis_lab as teal
from text_analysis_lab import translators as tr

p = teal.Project.create("study.teal", name="Study")
p = teal.Project.open("study.teal")

docs = p.read_csv(
    "docs.csv", text_fields="text",
    metadata_fields=["group", "date"], alias="documents")
docs = p.get_artifact("documents") # alias or ID
```

Also: `read_jsonl`, `read_parquet`, `read_excel`, and folder importers.

```
files = p.folder_inventory("corpus/", alias="files")
texts = p.translate(tr.TextFileExtractor(), files)["output"]
# PDFs: tr.PdfTextExtractor()
```

INSPECT / QUERY

```
docs.artifact_id      docs.aliases
docs.primary_key      docs.n_rows

docs.preview(20)
docs.table(limit=100)
docs.records(limit=100)
```

key_columns stable row identity data_columns data / representation
metadata_columns requested metadata
metadata_mode "none" | "local" | "full"

Selectors: True, False, one name, or a sequence. Metadata is opt-in.

```
docs.query(
    key_columns=True, data_columns=["text"],
    metadata_columns=["group"], metadata_mode="full",
    where="group = 'A'", order_by="date", limit=100)

docs.query_columns(metadata_mode="full")
```

TRANSLATE

```
out = p.translate(translator, source, alias="result")
result = out["output"]
```

Always returns an output-label->Artifact mapping. Pass whole Artifacts;
`batch_size=` is a request and `workers=` applies only when supported.

SAMPLE / CONTEXT / BATCH

```
docs.sample(n=25, random_state=42) # in memory
sample = p.sample(docs, n=500, alias="sample_500")

docs.get(key) docs.get_by_position(0)
docs.get_context(key, before=2, after=2)
docs.get_previous(key, n=3) docs.get_next(key, n=3)
docs.kwic("climate change", window=8)

for batch in docs.iter_table_batches(
    batch_size=5_000, data_columns=["text"]):
    ...
```

TeAL Cheat Sheet

Last edited: 2026-09-28

github.com/zachthinks/text-analysis-lab

STRUCTURAL OPERATIONS

```
p.sample(...) p.subset(...)
p.split(...) p.probability_split(...)
p.select_keys(...) p.restrict(...)
p.feature_subset(...) p.set_primary_keys(...)
p.aggregate(...) p.collapse_runs(...)
p.merge(...) p.join(...)
p.attach_metadata(...) p.from_keyed_frame(...)
```

Mappings: `translate`, `subset`, `split`, `probability_split`; most others return one Artifact.

GECO / HUMAN CODING

Explore: link a fixed document universe to a geometry and optional 2-D projections for interactive navigation, coding, and classifier development.

```
g = p.geco.create(
    "roomtemp", documents=docs, text_field="text",
    geometry=tfidf, projections={"umap": umap})
g.launch()
```

Add more representations or views later.

```
g.add_geometry("embeddings", embeddings)
g.add_projection("pca", pca, geometry="embeddings")
```

Bring human judgments back as a TeAL Artifact, or freeze a retained classifier for later prediction.

```
labels = g.export_codes("room temperature")
clf = g.export_classifier("room_temp_logistic")
```

Focus Coding: a finite, geometry-free annotation task over an exact document set. Export after all requested binary judgments are complete.

```
focus = p.geco.create_focus(
    "audit", documents=audit, text_source=docs,
    text_field="text", codes={"relevant": "Relevant"})
focus.launch_focus()
audit_labels = focus.export_focus_labels()
```

Manage linked workspaces: `p.geco.list()` `p.geco.open("roomtemp")`.

EXTERNAL RESULTS

```
pred = p.register_external(
    external_result, primary_key="document_id",
    data_fields=["prediction"],
    sources={"documents": docs}, alias="predictions")
```

`sources=` records operation provenance; `basis_labels=` + `lineage_mode=` declare structural basis.

```
scores = p.from_keyed_frame(
    docs, frame, data_fields=["score"], alias="scores")
```

Use `from_keyed_frame(...)` when measurements attach data to an existing key universe.

PROJECT CENTER / MEMOS

```
p.launch_project_center()
# CLI: teal gui path/to/project
```

Lineage: Artifact->Artifact. Provenance: Artifact(s)->Operation->Artifact(s). Inspect previews, storage, and versioned memos.

```
p.update_project_memo("Plan...", title="Analysis plan")
p.add_standalone_memo("Idea...", title="Idea")
```

VISUALIZATION

```
docs.visualize.histogram("text_words", by="group")

fig = docs.visualize.scatter(
    embeddings=umap, color="group",
    hover_fields=["doc_id", "text"],
    data_columns=["text"], metadata_columns=["group"],
    metadata_mode="full", sample_n=5_000)
fig.show()
```

Documents supply display fields; `embeddings=` supplies 2-D coordinates. Color/hover fields must be selected; query/sampling args apply.

ALIASES / RERUNS / PERSISTENCE

Artifacts are written directly to disk in the project folder.

```
lengths = p.translate(
    tr.TextLength({"text": ["words"]}), docs,
    alias="lengths")["output"]
```

If "lengths" already names a complete output, TeAL reuses it. Reuse is alias-based, not parameter-based.

```
# deliberately rebuild
... alias="lengths", overwrite=True
```

Overwrite may be blocked by other aliases or live dependents.

DELETE / PURGE / RESTORE

```
artifact.delete() # logical
p.delete_artifact("alias")
artifact.delete(recursive=True) # downstream closure
artifact.delete(recursive=True, purge_payload=True,
    memo="Superseded...")

artifact.is_deleted artifact.is_purged

d = p.get_artifact(artifact_id, include_deleted=True)
d.restore(new_alias="restored")
```

Delete is blocked if live downstream dependents would remain. Purge removes keys/, data/, metadata/ but keeps descriptor, provenance, and memos.

Purged Artifacts cannot be restored. Restore upstream dependencies before descendants; old aliases do not return automatically.

ANALYSIS

```
docs.analysis.summarize()
docs.analysis.text_diagnostics()
docs.analysis.crosstab("group", "label")

matrix.analysis.matrix_summary()
matrix.analysis.row_summary()
matrix.analysis.feature_summary()
matrix.analysis.distance(
    key=key1, other_key=key2, metric="euclidean")
embedding.analysis.nearest_neighbors(key=key, k=10)
embedding.analysis.cosine_similarity(key=key1, other_key=key2)
predictions.analysis.classification(gold=gold)
```

Distance metrics: "cosine", "euclidean"/"l2", "manhattan"/"l1"; dense matrices support additional compatible SciPy metrics.